# Popcorn Linux thread migration for compute intensive loads

Kimberly Devi Milner
CS 6233 Operating Systems, Summer 2021
NYU Tandon School of Engineeering
kimberly.milner@nyu.edu

***Abstract*— Popcorn Linux is a developing distributed kernel Operating System. It enables application as well as user space thread migration between kernels. We install the Operating system over heterogeneous machines and attempt to find use cases where performance is benefitted from thread migration between popcorn kernels.**

***Keywords—heterogeneous-Instruction Set Architecture (ISA) processors, shared memory programming, replicated kernel***

## I. Introduction

The Popcorn Linux OS is a developing Operating System that can run transparently over a heterogeneous set of kernels on heterogeneous architecture. Defining itself as a replicated-kernel architecture, each instance of the Popcorn Linux OS is handled as a distributed service. In this model multiple Linux kernel instances across heterogeneous hardware collaborate to present a consistent memory view of the Operating System.

The different kernels communicate entirely through message passing, and allow entire applications as well as user process threads to be "migrated" from one kernel to another; thread migrations occur at migration points. The Popcorn compiler generates machine code for both the x86 and ARM ISAs and instruments the code with migration points.

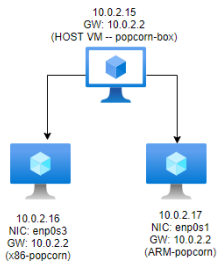
## II. Related Work

An effort to decouple the Operating System from the underlying hardware cores was attempted in the 2009 *Barrelfish* project, where intercore communication was made explicit; the distributed Operating System was made hardware neutral; and the Operating System state used asynchronous messaging for replication. [1] One Linux flavored efforts is Popcorn Linux, which creator Robert Lyerly explains in his theses was created at the same time as wider computing communities (e.g. high-performance compute) began to embrace CPU heterogeneity to realize energy efficiency and remove ISA restrictions existing on applications. [2]

## III. Methodology

### A. Setting up Popcorn Linux

We set up our Popcorn Linux environment on an Intel(R) Core(TM) i3-5015U CPU @ 2.10GHz 2.10 GHz with 8 GB of RAM. Using VirtualBox we created a host VM and two VMs for our popcorn instances.

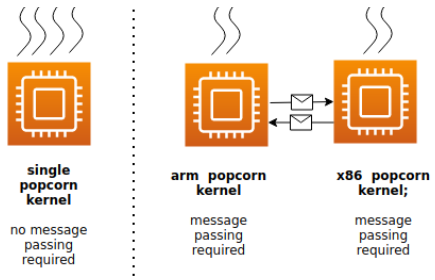



The host VM was installed in briged mode and the qemu emulator was invoked to run each popcorn kernl, one on x86 architecture and the other on ARM. We include additional technical notes for installation on our code repository. [4]

### B. Experiment

Migrating user process threads across kernels highly fascinated us as a load balancing paradigm.

With that in mind, we wanted to test a very simple hypothesis: Running N user pthreads over several popcorn-kernels we theorized should be more effective than running N user process pthreads on only one popcorn kernel for compute intensive jobs.



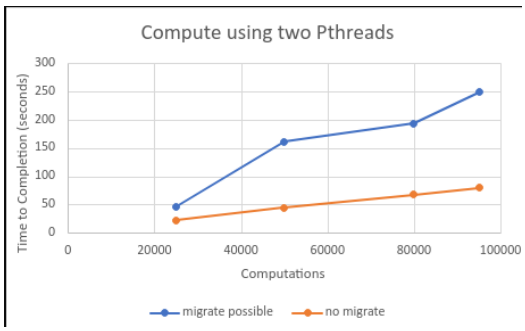

We simulated a "compute intensive" threaded job that just counted numbers, compiled it with the Popcorn Compiler, and ran it without any migrations on the ARM-popcorn VM. We compared these results against the job when it was allowed to migrate over kernels.

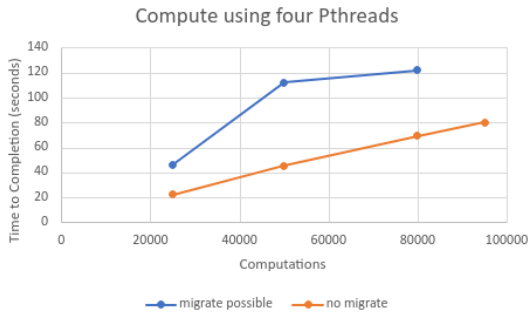
We acknowledge up front that our mechanism for forcing migrations was blunt, and involved just killing the popcorn thread, which then caused the thread to be run on the other VM kernel. This mechanism was encapsulated in the tutorial as a bash script scheduler and is what we used without modification.

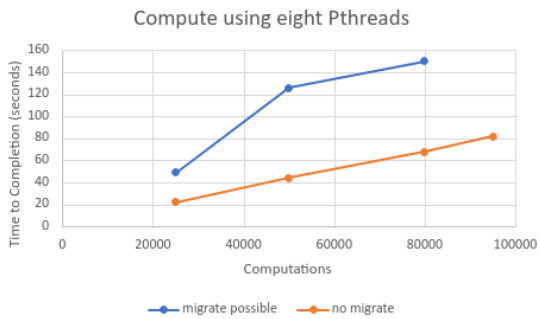
We varied the compute loads, as well as the number of threads used in the program, and ran the popcorn compiled program with the scheduler turned on and then off to understand how splitting load via thread migrations (albeit forced) could improve the performance of a compute intensive job.

## IV. Restuls and Observations

We did not observe the expected behavior. (Again, it is not impossible that our findings result from our machine setup being modestly spec'd.)




We made the following observations during testing:

1. The x86 machine consistently took longer to run the popcorn compiled number counting job

2. The recommended migration procedure corrupted the C program when dealing with larger compute loads

Initially we migrated threads manually, issuing a 35 signal to half of the created pthreads on the ARM-popcorn VM. When the compute load was large, the popcorn-compiled program on the ARM VM often went into a defunct state.

This behavior persisted even when using the thread migration scheduler provided in the tutorial.

When migration was enabled, the program became defunct on the ARM machine when the compute load exceeded 80K weighted operations. (Note the missing data points at 80K.)

```
root@ubuntu:~# ps -eaf | grep popcorn
root       1436    429 38 13:40 ?        00:06:49 [popcorn-prime_s] <defunct>
root       1975   1066 15 13:58 pts/0    00:00:00 grep --color=auto popcorn
```

## V. Conclusion

While we cannot precisely explain why the x86 VM ran more slowly than the ARM machine, we think it is safe to assume that having the migration scheduler run on the x68 VM could have contributed latency.

We also suspect that just killing a thread to force pthread migration was not sufficient to maintain program stability -- something occurred during the popcorn thread kill and/or popcorn thread migration that corrupted the running popcorn compiled program.

Nonetheless the model of distributed kernel programming seems promising, it is probable that thread migration conditions may require new coding and design paradigms.

It is also worth noting that the Popcorn architecture is being used by researchers as a platform to explore other aspects of multiprocessing applications. [5]

## VI. References

[1] R. Lyerly, "Popcorn linux: A compiler and runtime for state transformation between heterogeneous-ISA architectures"," Ph.D dissertation for Virginia Tech, 2016.

[2] P. Barham, "The Multikernel: A new OS architecture for scalable multicore systems," 2009.

[3] P. O. B. R. Antonio Barbalace, "Popcorn Linux Tutorial @SOSP2019," 2019.

[4] K. D. Milner, "Exploring the Popcorn Linux Operating System," rollingcoconut.github.io/popcorn-visitor.gitub.io, 2021.

[5] R. L. S. L. B. R. Ho-Ren Chuang, "Scaling Shared Memory Multiprocessing Applications in Non-cache-coherent Domains," in *13th ACM International Systems and Storage Conference (SYSTOR '20)*, 2020.